

BOOK 5 - AI ENGINEERING

Become an AI-Native Systems Engineer

Updated Detailed Curriculum with Weekly Scope Targets

Purpose of this updated edition

This version keeps the original 8-part AI engineering syllabus and adds a clear week-by-week execution plan, measurable weekly deliverables, monthly milestones, phase exit criteria, and a continuous portfolio capstone. The goal is not only to learn AI concepts, but to build, evaluate, operate, and explain production-grade AI systems.

Career target: a strong AI-native systems engineer who can integrate AI into production systems, design reliable workflows, debug AI behavior, operate AI-powered platforms, and think architecturally about AI.

Original roadmap strength	Updated roadmap improvement
Strong long-term direction toward AI-native engineering.	Adds weekly scope targets, deliverables, and done criteria.
Good 8-part syllabus structure.	Adds monthly milestones and phase reviews so progress is visible.
Good capstone idea: AI Operations Platform.	Turns the capstone into one continuous system that grows every phase.
Reliability and evaluation included later.	Moves evaluation, security, observability, and cost tracking into every phase.

Table of Contents

Section	What you will use it for
1. Purpose and career target	Clarifies what you are becoming and what this book is not trying to do.
2. Core philosophy and AI engineering stack	Keeps the original 8-part structure and learning priority order.
3. What changed in this update	Shows the practical improvements added to the original PDF.
4. Big-picture outcomes	Defines the 12-month and 18-month capability targets.
5. Weekly execution system	Defines how to study every week and what evidence to produce.
6. 64-week scope plan	The main week-by-week roadmap with targets and deliverables.
7. Continuous capstone	Explains how one portfolio project grows across all phases.
8. Part-by-part syllabus	Keeps the detailed curriculum topics grouped by the 8 original parts.
9. Reliability, security, and observability habits	Defines the engineering practices to repeat in every project.
10. Portfolio and interview narrative	Turns the learning into visible career proof.
11. Final review checklists	Simple checkpoints before moving to the next phase.

How to use this document

Treat the weekly plan as your operating system. Every week should end with one code commit, one note or README update, and one test, evaluation, or measurement. This prevents passive learning and creates portfolio evidence over time.

1. Purpose and Career Target

This curriculum is designed for engineers who already have software engineering background, Linux familiarity, backend understanding, debugging mindset, and some infrastructure exposure. That starting point is a strength. The goal is to convert that engineering base into AI-native capability.

This book is not designed to make you an ML researcher, a prompt engineer only, or someone who simply calls AI APIs. It is designed to help you become a strong AI-native systems engineer.

Meaning of the target role

Capability	What it means in practice
Understand AI systems	You can explain tokens, context, embeddings, retrieval, hallucination, structured outputs, tool calling, and model tradeoffs.
Build AI applications	You can build APIs and workflows that use LLMs reliably with schemas, retries, cost controls, and logs.
Design RAG systems	You can ingest documents, chunk them, index them, retrieve them, generate grounded answers, and evaluate quality.
Operate AI platforms	You can deploy, monitor, troubleshoot, and optimize AI services in production-like environments.
Build safe agents	You can design tool-using workflows with permissions, state, loop limits, approval gates, and auditability.
Think architecturally	You can explain tradeoffs across models, tools, data, latency, cost, reliability, security, and infrastructure.

What success looks like

- You can build a working AI backend, not just write prompts.
- You can explain why an AI system fails and how to improve it.
- You can evaluate AI output quality instead of trusting demos blindly.
- You can connect AI to real tools while keeping human control over risky actions.
- You can turn AI work into a portfolio that shows engineering depth.

2. Core Philosophy and AI Engineering Stack

Do not become overly dependent on AI, shallow in understanding, obsessed with hype, or tutorial-dependent. Become capable of building intelligent systems, integrating AI reliably, operating AI in production, and making tradeoffs like an engineer.

Part	Focus	Primary career value
Part 1	AI Foundations	Understand core concepts deeply enough to build with confidence.
Part 2	LLM Engineering	Build reliable LLM-backed APIs, services, and extraction systems.
Part 3	Prompt and Context Engineering	Control model behavior and manage context across workflows.
Part 4	RAG Systems	Build grounded knowledge systems over documents and engineering data.
Part 5	AI Agents and Workflows	Create safe tool-using workflows with state, approvals, and boundaries.
Part 6	AI Infrastructure	Serve, deploy, scale, route, and observe AI systems.
Part 7	AI Reliability and Evaluation	Measure quality, reduce failures, and add guardrails.
Part 8	AI Systems Architecture	Design enterprise AI platforms and production AI architectures.

Learning phase	Parts	Duration target	Big-picture outcome
Phase 1	Parts 1 and 2	Weeks 1-16	Foundations plus working LLM application engineering.
Phase 2	Parts 3 and 4	Weeks 17-36	Context-controlled workflows and production-style RAG.
Phase 3	Parts 5 and 6	Weeks 37-52	Safe agents, workflows, serving, gateways, and deployment.
Phase 4	Parts 7 and 8	Weeks 53-64	Evaluation, reliability, architecture, and capstone polish.

3. What Changed in This Updated Version

The original curriculum had the right direction. The main update is that the roadmap now has weekly scope targets and visible outputs. It also moves evaluation, security, observability, and cost discipline earlier, because those habits separate production engineers from demo builders.

Area	Original issue	Improvement added
Weekly scope	The PDF gave phases and examples, but not exact week-by-week targets.	Added a 64-week plan with scope, deliverable, and done criteria.
Capstone	The AI Operations Platform appeared mainly at the end.	Made it a continuous system that grows from Week 9 onward.
Evaluation	Evaluation was mostly in Part 7.	Added small eval datasets and regression checks from early weeks.
Security	Prompt injection and leakage were mentioned, but not repeated.	Added recurring AI threat modeling, tool boundaries, and approval gates.
Observability	Observability was listed in infrastructure.	Added logs, traces, token usage, latency, cost, and failure review as weekly habits.
Portfolio	Projects were listed separately.	Grouped projects into a coherent career story with GitHub proof, README notes, and demo milestones.

Rule for this updated roadmap

Do not count a week as complete because you watched videos or read documentation. Count it complete only when you have produced code, notes, and a measurable test, evaluation, or observation.

4. Big-Picture Outcomes

The roadmap has two outcome horizons: a 12-month practical engineering target and an 18-month architecture-level target. The 64-week plan is designed to reach the 12-month target first, then continue into reliability, architecture, and portfolio polish.

12-month target

Capability	Target by around Month 12
LLM fundamentals	Explain tokens, context windows, embeddings, temperature, hallucination, structured outputs, and model tradeoffs.
API engineering	Build LLM APIs with streaming, retries, schemas, cost tracking, prompt versioning, and logs.
RAG	Build a documentation assistant with ingestion, chunking, metadata, hybrid retrieval, re-ranking, citations, and evals.
Agents	Build controlled tool-using workflows with state, permissions, loop limits, and human approval.
AI infrastructure	Dockerize AI services, run local models, use queues, deploy basic Kubernetes manifests, and build an AI gateway.
Portfolio	Have 3-4 strong GitHub projects plus a growing AI Operations Platform capstone.

18-month target

Capability	Target by around Month 18
AI platform architecture	Design model routing, AI gateway, observability, auth, security, cost control, and governance.
Enterprise RAG	Design multi-source retrieval with metadata, permissions, citations, freshness, and evaluation.
Agentic workflows	Design safe automation with tools, state, approvals, rollback paths, and audit logs.
Production reliability	Use eval regression testing, monitoring, incident handling, prompt/model versioning, and failure analysis.
Career asset	Present a portfolio-grade AI Operations Platform with architecture diagrams, demo video, and tradeoff writeups.

5. Weekly Execution System

Use the original Thursday, Saturday, and Sunday rhythm, but make every week measurable. Each week should produce evidence that you can review later and show in interviews.

Day	Time target	Work target	Required output
Thursday	2-3 hours	Study the concept, run a small experiment, and debug your understanding.	Experiment code or notebook plus brief notes.
Saturday	2-4 hours	Integrate the weekly concept into a real project or the capstone.	Feature branch, code commit, or working prototype.
Sunday	45-60 minutes	Write architecture notes, tradeoffs, failure cases, and what to improve next.	README update, design note, or architecture diagram.
Daily optional	15-20 minutes	Read docs, AI engineering posts, GitHub repos, paper summaries, or try prompts.	One small insight captured in notes.

Definition of done for every week

Evidence	Minimum requirement
One code commit	A small but real artifact: CLI, API route, script, dashboard, worker, test, prompt harness, or infra file.
One note or README update	Explain what you learned, what failed, what changed, and what tradeoff you made.
One test, eval, or measurement	At minimum: a test case, sample eval dataset, latency measurement, token count, cost estimate, retrieval quality check, or failure log.

Do not skip the evidence step

The notes, tests, evals, and measurements are what turn learning into a career asset. They also make your capstone easier to explain because every design decision has a trail.

6. 64-Week Scope Target Plan

This is the main execution plan. Each row tells you what to focus on, what to build, and when the week is considered complete. The plan is intentionally practical: every phase ends in a useful project, and every project feeds the final AI Operations Platform capstone.

Pace guidance

If a week takes longer, extend it. Do not rush core engineering skills. It is better to complete 40 weeks deeply than 64 weeks shallowly. The weekly targets are a compass, not a race.

Phase 1 - Foundations and LLM Behavior (Weeks 1-8)

Goal: Understand how modern AI and LLM systems behave so you can build with confidence instead of copying examples.

Exit target: You can explain tokens, context, embeddings, hallucination, retrieval, and probabilistic model behavior using your own examples.

Week	Scope target	Build deliverable	Done when
1	AI landscape: AI vs ML vs deep learning vs generative AI, open-source vs closed models.	One-page AI engineering map.	You can explain where AI engineering fits in software systems.
2	LLM basics: tokens, tokenization, context windows, and next-token prediction.	Token counter CLI.	The tool counts tokens for sample prompts and shows context usage.
3	Generation controls: temperature, top_p, hallucination, reasoning limits, and prompt sensitivity.	Prompt behavior experiment notebook.	You compare outputs across settings and document failure patterns.
4	Embeddings and semantic similarity.	Mini semantic search over local notes.	Search returns reasonable results for at least 20 sample queries.
5	Vector search basics: cosine similarity, FAISS or Chroma concepts, and retrieval flow.	Embedding explorer.	You can inspect nearest neighbors and explain why matches appear.
6	Transformer intuition: attention, layers, scaling behavior, and why transformers changed AI.	Visual explanation notes.	You can explain transformer intuition without heavy math.
7	First evaluation habit: sample prompts, expected outputs, pass/fail review, and failure categories.	Simple eval script with 20 test prompts.	The script produces a pass/fail summary and failure notes.
8	Phase review and consolidation.	README or blog: How LLM apps actually work.	You summarize concepts, demos, limitations, and next steps.

Phase 2 - LLM Application Engineering (Weeks 9-16)

Goal: Build reliable LLM-backed backend services with schemas, retries, streaming, logging, and cost awareness.

Exit target: You have a production-style AI API backend that handles errors, validates outputs, tracks cost, and is covered by basic tests.

Week	Scope target	Build deliverable	Done when
9	LLM API basics: request lifecycle, messages, roles, model selection, and error responses.	Basic AI API endpoint.	Endpoint accepts input, calls an LLM, returns a clean response, and logs metadata.
10	Streaming responses, retries, timeouts, rate limits, and graceful failure handling.	Streaming AI backend.	Streaming works and retry behavior is tested with simulated failures.
11	Structured outputs, JSON schema, extraction use cases, and parse failures.	AI extraction pipeline.	Output validates against schema and rejects malformed responses.
12	Pydantic or schema validation, deterministic workflow steps, and typed outputs.	Resume, invoice, or ticket parser.	Parser extracts fields from examples and reports confidence or validation errors.
13	Tool/function calling: tool schemas, arguments, tool results, and tool boundaries.	Assistant that calls 2-3 real tools.	Tool calls are logged and invalid tool requests are rejected.
14	Cost control: token budgeting, prompt trimming, caching, and model choice.	Cost calculator middleware.	Each request records estimated cost, tokens, model, and latency.
15	Prompt versioning, regression tests, and prompt comparison.	Prompt test suite.	Two prompt versions are compared on the same eval set.
16	Phase project integration.	Production-style AI API backend.	Backend includes structured output, retries, logging, cost tracking, and tests.

Phase 3 - Prompt, Context, and Workflow Engineering (Weeks 17-24)

Goal: Control model behavior across multi-step workflows and reduce failures with validation and human review.

Exit target: You have a controlled AI document processor with validation, risk checks, logs, and human review for uncertain cases.

Week	Scope target	Build deliverable	Done when
17	Context engineering: context packing, memory boundaries, compression, and context freshness.	Context builder utility.	Utility selects and formats only relevant context for a task.
18	Prompt architecture: system instructions, developer constraints, examples, and output contracts.	Prompt template library.	Templates are versioned and include purpose, inputs, outputs, and tests.
19	Multi-step workflows: classify, retrieve, generate, validate, and escalate.	Multi-stage AI pipeline.	Pipeline exposes each step result and failure reason.
20	Human-in-the-loop patterns: review queues, approvals, and escalation paths.	Review and approval queue.	Uncertain or risky outputs are routed to human review.
21	Prompt injection and data leakage basics.	Prompt-injection test cases.	At least 15 adversarial inputs are tested and documented.
22	Output validation, safe formatting, and downstream system protection.	Guarded output pipeline.	Invalid outputs fail closed and do not reach downstream actions.
23	Workflow observability: trace IDs, step logs, token usage, cost, and error categories.	AI request log dashboard.	Dashboard shows inputs summary, steps, model, latency, cost, and errors.
24	Phase project integration.	Controlled AI document processor.	Processor extracts data, validates it, detects risky input, and supports review.

Phase 4 - RAG Systems (Weeks 25-36)

Goal: Build a retrieval-augmented generation system that answers from documents with citations and measurable retrieval quality.

Exit target: You have an engineering documentation chatbot with ingestion, chunking, metadata, hybrid search, re-ranking, citations, and evaluation.

Week	Scope target	Build deliverable	Done when
25	RAG foundations: why RAG exists, retrieval flow, grounding, and answer constraints.	RAG architecture diagram.	Diagram shows ingestion, indexing, retrieval, generation, citations, and eval.
26	Document ingestion: PDF, Markdown, HTML, text extraction, cleaning, and metadata.	Ingestion pipeline.	Pipeline ingests at least 20 documents and stores metadata.
27	Chunking strategies: fixed, semantic, section-based, overlap, and metadata tradeoffs.	Chunking experiment report.	You compare at least 3 chunking strategies with sample queries.
28	Vector database basics: Chroma, FAISS, Pinecone concepts, indexing, and filtering.	Search API over documents.	API returns top chunks with scores and metadata.
29	Hybrid retrieval: keyword plus vector search, filters, and query rewriting.	Hybrid retrieval prototype.	Hybrid search beats vector-only on at least some test cases.
30	Re-ranking and retrieval quality analysis.	Reranker experiment.	You compare top-k retrieval before and after re-ranking.
31	Grounded generation: citations, answer constraints, refusal when context is missing.	Documentation Q&A bot.	Answers include source snippets or citations and avoid unsupported claims.
32	RAG evaluation: answer correctness, citation accuracy, retrieval recall, and failure types.	50-question RAG eval set.	Eval report includes pass rate and top failure categories.
33	RAG observability: retrieved chunks, latency, cost, query logs, and hallucination review.	RAG trace viewer.	Each answer can be traced to retrieved chunks and model output.
34	Incremental indexing: document updates, deletes, re-indexing, and freshness.	Auto-update document index.	Changed documents update without rebuilding the full index.
35	Advanced RAG: multi-source retrieval, graph intuition, long context, and agentic RAG.	Advanced RAG design document.	Design explains when to use each pattern and its tradeoffs.
36	Phase project integration.	Engineering documentation chatbot.	Chatbot supports ingestion, hybrid search, citations, evals, and traces.

Phase 5 - Agents and Workflows (Weeks 37-44)

Goal: Build safe tool-using AI workflows with permissions, state, loop limits, approval gates, and auditability.

Exit target: You have an AI deployment analyzer that can inspect information, suggest actions, and require approval before risky operations.

Week	Scope target	Build deliverable	Done when
37	Agent fundamentals: agent loop, planning, tools, observations, state, and stop conditions.	Simple tool-using agent.	Agent can use a small tool set and stops predictably.
38	Tool boundaries and permissions: allow lists, deny lists, scopes, and action risk levels.	Tool registry with rules.	Each tool has schema, permission level, owner, and logging.
39	Agent state and memory: task state, conversation memory, retrieval memory, and persistence.	Stateful workflow assistant.	Assistant resumes a task from stored state without unsafe assumptions.
40	Workflow agents vs autonomous agents: deterministic orchestration and controlled decisions.	Deterministic workflow orchestrator.	Workflow uses explicit steps rather than open-ended autonomy.
41	Connector and MCP-style thinking: resources, tools, schemas, and external context interfaces.	Connector-style resource/tool interface.	Interface exposes resources and tools with clear schemas and limits.
42	Agent reliability: retries, loop limits, timeouts, fallback, and failure classification.	Agent safety wrapper.	Wrapper prevents infinite loops and records failure reasons.
43	Human approval for risky actions: previews, confirmations, audit logs, and rollback notes.	Approval-based DevOps assistant.	Assistant proposes actions but cannot execute risky changes without approval.
44	Phase project integration.	AI deployment analyzer.	Analyzer reads logs/configs, summarizes risks, recommends actions, and asks approval.

Phase 6 - AI Infrastructure and Serving (Weeks 45-52)

Goal: Operate AI systems like a production engineer with serving, queues, deployment, routing, monitoring, and cost controls.

Exit target: You have an AI gateway service that routes requests, logs model usage, tracks cost, supports retries, and exposes monitoring.

Week	Scope target	Build deliverable	Done when
45	Model serving overview: API providers, local models, vLLM, Ollama, TGI, and serving tradeoffs.	Serving comparison notes.	You can explain when to use hosted APIs vs local/self-hosted models.
46	Run a local model with Ollama or a similar local serving tool.	Local LLM endpoint.	Local endpoint accepts prompts and is called from your app.
47	Dockerizing AI services: app image, dependencies, env vars, and reproducibility.	Dockerized AI backend.	Service runs locally from Docker with documented commands.
48	Queue-based AI processing: background jobs, retries, idempotency, and long-running tasks.	Async job worker.	Worker processes jobs and records status, failure, and retry count.
49	Kubernetes basics for AI apps: deployments, services, config, secrets, and scaling concepts.	Kubernetes deployment manifest.	Manifest deploys your service in a local or test cluster.
50	AI gateway: routing, rate limits, auth, model selection, and cost controls.	Basic AI gateway.	Gateway routes to at least two model backends or modes.
51	Observability: metrics, traces, logs, token/cost dashboard, and failure analysis.	Monitoring dashboard.	Dashboard shows latency, token usage, cost, errors, and model usage.
52	Phase project integration.	AI inference deployment system.	System includes gateway, worker, logs, monitoring, and deployment docs.

Phase 7 - Reliability, Evaluation, and Guardrails (Weeks 53-58)

Goal: Make AI systems trustworthy by measuring quality, detecting regressions, handling failures, and controlling risky outputs.

Exit target: You have an AI evaluation dashboard showing pass rate, hallucination risk, retrieval quality, latency, and cost.

Week	Scope target	Build deliverable	Done when
53	Evaluation fundamentals: datasets, graders, regression tests, and test-set versioning.	Eval harness.	Harness runs repeatable evals and stores results by version.
54	RAG evaluation: groundedness, citation quality, retrieval recall, and answer usefulness.	RAG eval dashboard.	Dashboard shows retrieval and answer quality across test queries.
55	Guardrails: input validation, output validation, safety filters, and schema boundaries.	Guardrail middleware.	Middleware blocks invalid or unsafe input/output patterns.
56	Failure handling: retry, fallback, escalation, timeouts, and user-visible error states.	Reliability wrapper.	Wrapper classifies failures and chooses safe fallback behavior.
57	Cost and performance optimization: caching, compression, routing, batching, and model choice.	Optimization report.	Report compares cost, latency, and quality before and after optimization.
58	Phase project integration.	AI quality analyzer.	Analyzer reports pass rate, failures, latency, cost, and improvement recommendations.

Phase 8 - AI Systems Architecture and Capstone (Weeks 59-64)

Goal: Integrate everything into a portfolio-grade AI Operations Platform and explain it like a systems architect.

Exit target: You have a polished capstone with code, demo, architecture diagrams, evaluation results, and a clear interview narrative.

Week	Scope target	Build deliverable	Done when
59	AI system design patterns: gateways, orchestrators, RAG services, workers, eval services, and audit logs.	Architecture decision records.	ADRs explain key choices and alternatives.
60	Enterprise RAG and permissions: multi-tenant design, metadata, access control, and data boundaries.	Multi-tenant RAG design.	Design explains how user permissions affect retrieval and answers.
61	Distributed AI workflows: queues, workers, streaming, retries, backpressure, and failure domains.	Distributed architecture diagram.	Diagram shows services, data flow, failure points, and scaling paths.
62	AI incident handling and operational reliability.	AI incident playbook.	Playbook covers hallucinations, bad retrieval, tool failure, latency spikes, and cost spikes.
63	Capstone integration: combine RAG, tools, agents, evaluation, gateway, observability, and deployment.	AI Operations Platform MVP.	MVP supports log analysis, runbook retrieval, recommendations, approvals, and dashboarding.
64	Portfolio polish: README, demo video script, architecture deck, screenshots, and final review.	Published portfolio package.	Repository explains problem, architecture, setup, demo, evals, failures, and tradeoffs.

7. Continuous Capstone: AI Operations Platform

Instead of building many disconnected projects, use one long-running capstone that grows every phase. The final system should demonstrate LLM engineering, RAG, agents, infrastructure, evaluation, security, and architecture in one coherent portfolio story.

Phase	Capstone growth target
Weeks 1-8	Build small standalone utilities: token counter, embedding explorer, semantic search. These become learning tools and later support components.
Weeks 9-16	Create the first AI API backend with structured outputs, retries, logs, and cost tracking.
Weeks 17-24	Add controlled workflows, prompt templates, validation, approval queues, and prompt-injection tests.
Weeks 25-36	Add RAG over engineering docs, runbooks, logs, and incident notes with citations and evals.
Weeks 37-44	Add safe agent workflows that inspect logs/configs, recommend actions, and ask approval before risky operations.
Weeks 45-52	Add AI gateway, queue worker, Docker/Kubernetes deployment, and monitoring.
Weeks 53-58	Add eval dashboard, guardrails, failure analysis, quality scoring, and optimization reports.
Weeks 59-64	Integrate, polish, document, record demo, and prepare architecture narrative.

Final capstone capabilities

Capability	Included feature
Log analysis	Upload logs and receive a structured diagnosis with likely causes and next checks.
RAG	Search runbooks, docs, incident notes, and architecture notes with cited answers.
Tool calling	Query safe status APIs, configs, deployments, and metadata.
Agent workflow	Diagnose, suggest, ask approval, and only then execute low-risk actions.
Evaluation	Run regression tests for extraction, RAG answers, tool use, and refusal behavior.
Observability	Track cost, latency, token usage, retrieved chunks, tool calls, and failures.
Infrastructure	Dockerized app, queue worker, gateway, monitoring, and Kubernetes manifests.
Security	Prompt-injection tests, tool permissions, data boundaries, and audit logs.
Architecture	System diagrams, ADRs, tradeoff explanations, and incident playbook.

8. Part-by-Part Detailed Syllabus

This section preserves the original 8-part structure, but makes the build outcomes clearer. Use it as a syllabus reference while the 64-week plan acts as your execution checklist.

Part	Goal	Core topics	Build outcomes
1. Foundations	Understand AI concepts deeply enough to build confidently.	AI landscape; neural network intuition; tokens; context windows; next-token prediction; temperature; hallucinations; embeddings; semantic similarity; transformer intuition.	Token counter; semantic similarity search; embedding explorer; evaluation habit.
2. LLM Eng.	Build applications using modern LLMs.	LLM APIs; structured outputs; JSON mode; function/tool calling; streaming; retries; cost optimization; open-source models; local models; schema validation.	AI CLI assistant; AI API backend; extraction pipeline; tool-calling assistant.
3. Prompt/Context	Control AI system behavior effectively.	Context management; context injection; memory; compression; prompt architecture; multi-step workflows; validation; human-in-loop; prompt injection; data leakage.	Multi-stage workflow; AI document processor; controlled assistant; guardrail pipeline.
4. RAG	Master retrieval-augmented generation deeply.	RAG foundations; chunking; search pipelines; vector databases; hybrid search; re-ranking; production RAG; incremental indexing; advanced RAG patterns.	Engineering docs chatbot; knowledge system; GitHub repo assistant; troubleshooting assistant.
5. Agents/Workflows	Learn orchestration and safe AI automation.	Agent loops; planning; tool usage; workflow agents; memory; state; event-driven automation; reliability; tool misuse; human override.	AI DevOps assistant; deployment analyzer; workflow orchestrator; autonomous debugging prototype with controls.
6. Infrastructure	Operate AI systems like a production engineer.	Model serving; local models; GPU/VRAM intuition; quantization; Docker; Kubernetes; scaling; AI observability; gateways; routing.	Self-hosted AI platform; AI gateway; inference deployment system; Kubernetes AI stack.
7. Reliability/Eval	Build trustworthy AI systems.	Eval datasets; usefulness vs accuracy; hallucination measurement; retry systems; confidence scoring; guardrails; human review; cost and performance optimization.	Evaluation dashboard; guardrail system; AI quality analyzer; reliability wrapper.
8. Architecture	Think like an AI systems architect.	AI architecture patterns; microservices; orchestration layers; enterprise RAG; distributed inference; queue-based AI systems; streaming; AI incident handling; future trends.	AI Operations Platform capstone; architecture diagrams; incident playbook; portfolio package.

9. Monthly Milestone Targets

Use this table for high-level progress review. If you miss a monthly milestone, slow down and finish the project before moving on. Depth matters more than speed.

Month	Milestone
1	Understand LLM basics, tokens, context windows, generation controls, and hallucination patterns.
2	Build semantic search, embedding explorer, and first eval script.
3	Build reliable LLM API with structured output and streaming.
4	Build tool-calling assistant and cost tracking middleware.
5	Build controlled AI workflow with context builder and prompt templates.
6	Build document processor with validation, prompt-injection tests, and review queue.
7	Build RAG ingestion, chunking, vector search, and metadata filtering.
8	Build production-style RAG chatbot with citations and eval dataset.
9	Build safe agent workflow with tools, state, permissions, and approval gates.
10	Build deployment analyzer and AI workflow orchestrator.
11	Build local model endpoint, Dockerized service, async worker, and Kubernetes manifest.
12	Build AI gateway and observability dashboard with latency, token, cost, and failure metrics.
13	Build eval harness, RAG eval dashboard, guardrails, and reliability wrapper.
14	Optimize cost/performance and create AI quality analyzer.
15	Integrate capstone architecture, enterprise RAG design, and incident playbook.
16	Polish GitHub, README, demo, architecture deck, screenshots, and interview narrative.

10. Reliability, Security, and Observability Habits

These are recurring engineering habits. Start small, but repeat them throughout the roadmap. The goal is to become someone who can trust, debug, and operate AI systems instead of only demoing them.

Evaluation habit

Artifact	Minimum standard
Test dataset	Start with 20 prompts or queries; grow to 50-100 for important systems.
Expected behavior	Define success criteria: correct fields, grounded answer, valid JSON, safe refusal, or correct tool use.
Failure log	Record hallucination, bad retrieval, bad tool call, parse failure, timeout, high cost, or unsafe output.
Versioning	Track prompt version, model version, eval set version, and notable config changes.

Security habit

Question	Why it matters
Can the user override system instructions?	Prompt injection and instruction hierarchy testing.
Can the model reveal private or hidden data?	Sensitive information disclosure and data leakage prevention.
Can a tool perform a risky action?	Excessive agency, missing authorization, and unsafe automation.
Can generated output break downstream systems?	Insecure output handling, command injection, malformed JSON, or unsafe SQL.
Can retrieved documents poison the answer?	RAG data poisoning, stale docs, low-quality sources, or conflicting evidence.

Observability habit

Metric or log	Why to track it
Prompt and completion tokens	Cost control and context growth analysis.
Latency	User experience, timeouts, and system bottlenecks.
Model name and version	Reproducibility and regression analysis.
Retrieved documents/chunks	RAG debugging and citation verification.
Tool calls and tool results	Agent safety, auditability, and failure investigation.
Eval pass/fail	Quality trend and regression detection.
Human override or approval	Governance and safe automation tracking.

11. Portfolio and Interview Narrative

Your portfolio should prove that you can build AI systems, not just follow tutorials. Each project should have a clear problem, architecture, setup instructions, screenshots, tests, and a short explanation of tradeoffs.

Portfolio asset	What it should prove
AI API backend	You can build a reliable LLM service with structured outputs, retries, streaming, logs, tests, and cost tracking.
Controlled document processor	You can manage context, validate outputs, detect risky inputs, and use human review.
RAG documentation chatbot	You can build ingestion, chunking, hybrid search, re-ranking, citations, and quality evaluation.
AI deployment analyzer	You can design safe tool-using workflows with state, permissions, and approval gates.
AI gateway and monitoring stack	You can route model requests, deploy services, observe token/cost/latency, and operate AI workloads.
AI Operations Platform capstone	You can integrate RAG, tools, agents, evals, guardrails, observability, deployment, and architecture.

Interview story template

Use this narrative

I built an AI Operations Platform end to end. It started as an LLM API, then I added structured outputs, context control, RAG over engineering documents, safe tool-calling workflows, evaluation, guardrails, cost tracking, observability, and deployment. I can explain the architecture, tradeoffs, failure modes, and how I measured quality.

Repository README checklist

README section	Include
Problem	What real engineering pain the project solves.
Architecture	Diagram plus service/data/tool flow.
Setup	Commands to run locally with Docker or simple scripts.
Demo	Screenshots, sample inputs, sample outputs, and optional video link.
Evaluation	Dataset, metrics, examples of passing and failing cases.
Reliability	Retries, timeouts, validation, fallbacks, and known limitations.
Security	Prompt-injection tests, permission boundaries, approval gates, and data handling notes.
Tradeoffs	Why you chose specific models, retrieval strategy, architecture, and cost controls.

12. Final Review Checklists

Use these checklists to decide whether to move forward. You do not need perfection, but you do need working evidence and enough understanding to explain your design decisions.

Phase	Move forward only when you can do this
After Weeks 1-8	Explain LLM behavior, tokens, embeddings, retrieval, and hallucination using your own experiments.
After Weeks 9-16	Build and test an AI backend with schema validation, retries, streaming, logs, and cost tracking.
After Weeks 17-24	Design a controlled AI workflow with validation, context control, risk checks, and human review.
After Weeks 25-36	Build a RAG system that retrieves useful context, cites sources, and has a measurable eval set.
After Weeks 37-44	Build a safe tool-using workflow with permissions, state, loop limits, and approval gates.
After Weeks 45-52	Deploy or containerize AI services with gateway, worker, monitoring, and infrastructure notes.
After Weeks 53-58	Run repeatable evals and explain quality, cost, latency, reliability, and failure categories.
After Weeks 59-64	Present the capstone clearly with code, demo, architecture, evals, tradeoffs, and limitations.

What to avoid

- Do not become a prompt-only engineer.
- Do not become an AI wrapper developer who cannot debug failures.
- Do not copy-paste AI demos without understanding the architecture.
- Do not ignore evaluation, cost, security, and observability.
- Do not rush the roadmap just to finish a document.

Final message

The strongest AI engineers will not necessarily be the best ML researchers or the best prompt writers. They will be engineers who can integrate AI into systems, operate AI reliably, debug AI workflows, scale AI platforms, and architect intelligent production systems. That is the exact purpose of this updated roadmap.